

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y
Ciencias de la Computación

ANÁLISIS DE SISTEMAS II

Ing. Richard David Ortiz Sasvin



MICRO-HIS: TRASLADOS, ALTAS Y LIBERACIÓN DE CAMAS Microservicios y Monolito

Nombre: Esaú Abimael de la Cruz Mendoza

No. de Carné: 1890-21-13279

Correo: edelacruz@miumg.edu.gt

Repositorio: github.com/00AbiMendoza/micro-his-asii-08-traslados-altas-camas

Rama: feature/micro-his-asii-08

Etiqueta: entrega-micro-his-asii-08-v1.0

Commit de evidencia: 68cd66c

Fecha: 19 / 08 / 2026

Guastatoya, El Progreso, Guatemala

Índice

Introducción	1
Desarrollo del Micro-HIS	2
Arquitectura por capas	3
Flujo de traslado	4
Alta y reglas de dominio	5
Persistencia y transacciones	6
Pruebas automatizadas	7
Evidencias, Git, uso de IA y defensa	8
Conclusión	9
Bibliografía	10

Introducción

Esta actividad parte de una idea sencilla: construir un pequeño módulo hospitalario que realmente se pueda ejecutar y comprobar. El proyecto desarrollado se llama «Micro-HIS Traslados, altas y liberación de camas» y fue realizado en PHP 8.2 o superior, sin utilizar frameworks (PHP Documentation Group, 2026a). La intención no fue crear un sistema hospitalario completo, sino concentrarse en un flujo concreto y aplicar una organización de código que permita entender con claridad dónde se encuentra cada responsabilidad.

Para lograrlo, el proyecto se separó en cuatro capas: Presentation, Application, Domain y Persistence. Esta división permite que las reglas relacionadas con las camas y las admisiones no dependan directamente de la consola ni de la base de datos. También se utilizó PDO con sentencias preparadas, configuración externa y transacciones para evitar que un traslado o un alta queden guardados a medias si ocurre un error.

El trabajo se complementó con datos totalmente ficticios, pruebas automatizadas, diagramas editables y evidencias de ejecución y de Git. De esta manera, la entrega no se limita a describir una solución, sino que muestra que el flujo funciona y que las decisiones tomadas se pueden explicar y defender.

Desarrollo del Micro-HIS

Descripción general

El módulo asignado corresponde a traslados, altas y liberación de camas. Para el ejercicio se trabajó con una admisión ficticia y tres camas identificadas con códigos igualmente ficticios. El caso principal consiste en mover una admisión activa desde una cama ocupada hacia otra disponible, dejando la cama anterior en estado de limpieza.

Objetivo funcional

El objetivo es que el traslado o el alta mantengan la información consistente. En un traslado, la admisión continúa activa, cambia de cama y la cama de origen no queda disponible de inmediato. En un alta, la admisión se cierra y la cama utilizada también pasa a limpieza. Esta decisión evita representar una cama como utilizable antes de completar el proceso de saneamiento.

Entorno utilizado

- PHP 8.4.24 CLI, compatible con el requisito de PHP 8.2 o superior (PHP Documentation Group, 2026a).
- Aplicación PHP vanilla, sin framework.
- SQLite para una ejecución local sencilla y reproducible.
- PDO con sentencias preparadas para el acceso a datos (PHP Documentation Group, 2026b).
- Datos de prueba como PAC-FICT-001 y CAMA-FICT-A101, sin información clínica real.

Arquitectura por capas

La estructura se organizó en cuatro capas para que la lógica del módulo no quede mezclada con la entrada por consola ni con SQLite. En la práctica, esto facilita leer el proyecto y también permite probar los casos de uso sin depender de una base de datos real.

Tabla 1

Capas de la arquitectura y su responsabilidad

Capa	Responsabilidad	Ejemplos
Presentation	Recibe los comandos y muestra el resultado al usuario.	ConsoleApplication
Application	Coordina el traslado, el alta y las transacciones.	TransferPatient, DischargePatient
Domain	Contiene entidades, reglas y contratos del negocio.	Bed, Admission, Repositories
Persistence	Implementa los contratos con PDO y SQLite.	Pdo*Repository, PdoTransactionManager

Decisión principal

Application depende de contratos y no de implementaciones concretas de PDO. Por eso, los repositorios definidos en Domain pueden ser reemplazados por dobles de prueba. El diagrama editable de esta organización se encuentra en docs/diagramas/arquitectura-micro-his.puml y su versión visual en arquitectura-micro-his.png.

Flujo de traslado

El traslado se diseñó como una operación completa. Antes de modificar datos se revisa que la admisión exista, que siga activa, que la cama de origen exista y que la cama destino sea diferente y esté disponible.

- Se recibe el identificador de la admisión y de la cama destino.
- Se cargan la admisión, la cama actual y la cama destino.
- Se comprueba que la cama destino tenga estado available.
- Se inicia una transacción.
- La cama de origen cambia de occupied a cleaning.
- La cama destino cambia de available a occupied.
- La admisión mantiene el estado active y cambia su bedId.
- Se guardan los cambios y se ejecuta commit.
- Si ocurre una excepción durante la persistencia, se ejecuta rollback.

En la ejecución realizada, la admisión 1 pasó de la cama 1 a la cama 2. El resultado posterior mostró la cama 1 en cleaning, la cama 2 en occupied y la admisión todavía activa, ahora asociada a la cama 2. Este resultado quedó guardado como evidencia dentro del repositorio.

Alta y reglas de dominio

Flujo de alta

El alta utiliza la misma idea de consistencia. Primero se comprueba que la admisión siga activa y se obtiene la cama asociada. Después se inicia una transacción, la cama pasa a cleaning, la admisión cambia a closed y los cambios se confirman únicamente si toda la operación termina correctamente.

Reglas aplicadas

- Una cama solo puede ocuparse cuando está available.
- Una cama ocupada no pasa directamente a available; primero debe quedar en cleaning.
- Una cama solo puede volver a available después de la etapa de limpieza.
- No se permite trasladar una admisión hacia la misma cama.
- No se puede trasladar ni dar de alta una admisión que ya esté cerrada.

Estas reglas se encuentran en las entidades y en los casos de uso, no en la interfaz de consola. Con esto se busca que el comportamiento del módulo siga siendo el mismo aunque en el futuro cambie la forma de entrada o el mecanismo de persistencia.

Persistencia y transacciones

PDO y SQLite

La capa Persistence utiliza una conexión PDO configurada para trabajar con SQLite. Los repositorios `PdoBedRepository` y `PdoAdmissionRepository` realizan las consultas y actualizaciones mediante sentencias preparadas (PHP Documentation Group, 2026b). De esta forma, los valores se envían como parámetros y no se construyen consultas concatenando directamente datos de entrada.

Configuración externa

La ruta de la base de datos y el entorno se obtienen desde un archivo `.env`. En el repositorio solamente se conserva `.env.example`, mientras que el archivo real y la base `micro_his.sqlite` están excluidos mediante `.gitignore`. Esta separación evita subir configuración local o archivos generados durante la ejecución.

Transacciones

`PdoTransactionManager` concentra `begin`, `commit` y `rollback`. Los casos de uso inician la transacción antes de cambiar el estado persistente. Si todo se guarda correctamente se confirma la operación; si falla algún repositorio, se revierte. Esta decisión es importante porque un traslado involucra varias modificaciones que deben comportarse como una sola unidad.

Pruebas automatizadas

Las pruebas se implementaron en tests/run.php sin utilizar un framework. Para los casos de Application se usaron repositorios y un administrador de transacciones falsos, lo que permite comprobar las reglas sin depender de SQLite.

Tabla 2

Pruebas automatizadas ejecutadas y resultado

Prueba	Qué valida	Resultado
Traslado exitoso	Comprueba cama origen, destino, admisión y commit.	OK
Destino ocupado	Verifica que la regla rechace el traslado antes de iniciar la transacción.	OK
Error de persistencia	Simula un fallo y comprueba rollback sin commit.	OK
Alta exitosa	Comprueba cierre de admisión y cama en cleaning.	OK

La ejecución final mostró «4 aprobadas, 0 fallidas». Con estas pruebas se cubren los tres puntos solicitados en la consigna: un camino feliz, una regla de dominio y un error de persistencia. Además, se agregó una prueba de alta para tener evidencia de los dos procesos principales del módulo.

Evidencias, Git, uso de IA y defensa

Evidencias de ejecución y Git

El repositorio conserva archivos con el resultado de las pruebas, la ejecución real del traslado, el estado posterior de las camas, el historial git log --oneline y el listado de archivos. Antes de preparar el informe, el historial llegaba al commit 68cd66c, correspondiente a las evidencias de ejecución y validación.

- Repositorio: github.com/00AbiMendoza/micro-his-asii-08-traslados-altas-camas
- Rama de trabajo: feature/micro-his-asii-08
- Etiqueta prevista para la entrega evaluada: entrega-micro-his-asii-08-v1.0

Uso de inteligencia artificial

DECLARACION_IA.md registra el uso de ChatGPT como apoyo para la estructura, revisión de código, pruebas, documentación y diagramas. También deja constancia de que las propuestas fueron ejecutadas y verificadas manualmente, incluyendo correcciones realizadas durante el proceso.

Defensa individual

GUIA_DEFENSA.md resume el problema, las cuatro capas, las reglas principales, el uso de transacciones y las pruebas. La idea es poder explicar por qué la cama pasa a cleaning, cómo se evita un guardado parcial y qué se modificaría si el docente solicita un cambio sencillo durante la defensa.

Conclusión

El ejercicio permitió construir un módulo pequeño pero completamente ejecutable para traslados, altas y liberación de camas. La separación entre Presentation, Application, Domain y Persistence ayudó a mantener las reglas del negocio fuera de la consola y de PDO. La decisión que considero más importante fue manejar el traslado y el alta mediante transacciones, ya que ambas operaciones modifican varios datos que deben quedar consistentes entre sí.

Las pruebas y la ejecución con SQLite demostraron que el flujo principal funciona, que una cama destino ocupada es rechazada y que un error de persistencia provoca rollback. Al mismo tiempo, el uso de datos ficticios permitió trabajar sin exponer información clínica real.

Como limitación, esta versión utiliza una interfaz de consola y SQLite porque el objetivo es académico. En un sistema hospitalario de producción todavía harían falta autenticación, autorización, auditoría, control de concurrencia y una infraestructura de base de datos más completa. Aun así, el código, los diagramas editables, las pruebas y las evidencias Git permiten comprobar de forma clara lo realizado.

Bibliografía

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. <https://www.omg.org/spec/UML/2.5.1>

PHP Documentation Group. (2026a). PHP: Supported versions. <https://www.php.net/supported-versions.php>

PHP Documentation Group. (2026b). PDO — PHP Data Objects. PHP Manual. <https://www.php.net/manual/en/book.pdo.php>

PHP Documentation Group. (2026c). PDO::prepare. PHP Manual. <https://www.php.net/manual/en/pdo.prepare.php>